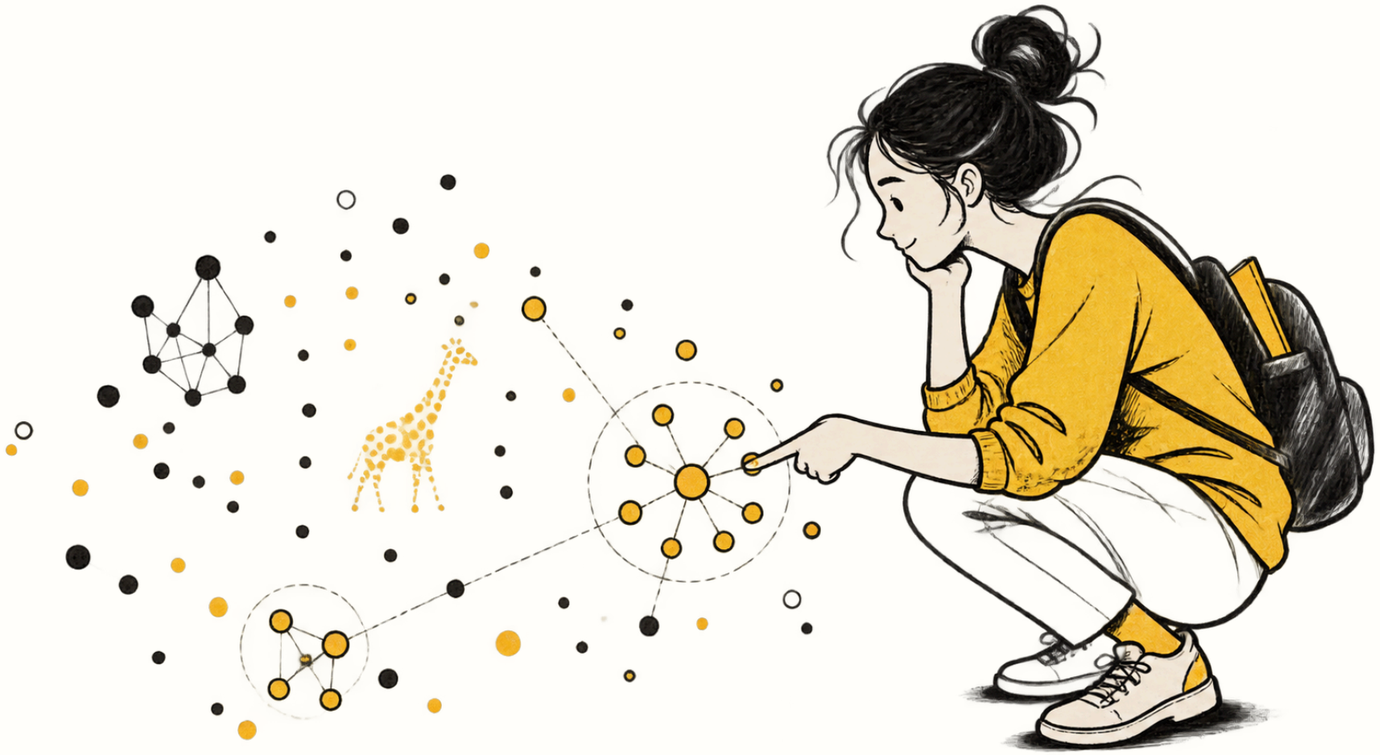
Нейронные сети и свёртки

От похожих объектов – к локальным шаблонам и вектору признаков



girafe ai Радослав Нейчев

Продолжим менять представление

$$\mathbb{X} \to \mathbb{X}_1 \to \mathbb{X}_2 \to \cdots \to \mathbb{Y}$$

$$\mathbf{x} \longmapsto \mathbf{x}_1 \longmapsto \mathbf{x}_2 \longmapsto \hat{y}$$

Теперь преобразование использует соседство пикселей и общие веса.

Сначала обучим новые признаки, затем используем устройство изображения.

Что осталось с предыдущей лекции?

Независимость наблюдений ≠ независимость признаков.

Линейный классификатор: одна граница. kNN: сравнение с обучающими объектами.

Отступ отвечает на вопрос: насколько уверенно объект расположен относительно границы?

$$y \in \{-1, +1\}, \qquad m = y(\mathbf{w}^\mathsf{T}\mathbf{x} + b)$$

Геометрический отступ получается после деления на $\|\mathbf{w}\|_2$.

От one-hot – к обучаемой таблице

Слово → Индекс строки → Вектор признаков

One-hot-векторы разных слов ортогональны. Эмбеддинги могут отражать сходство контекстов.

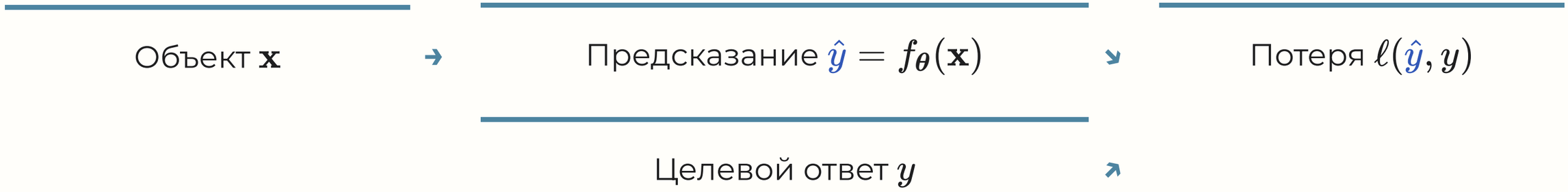
Вектор слова берём из таблицы по индексу – полный one-hot хранить не нужно.

Пары «слово – контекст» получаем из текста без ручной разметки. Частоты слов несбалансированы.

Что значит «похожие объекты»?

Выбираем пространство объектов $\mathbb{X}$ и пространство ответов $\mathbb{Y}$.

$$f_{\boldsymbol{\theta}} : \mathbb{X} \to \mathbb{Y}$$



Потеря оценивает расхождение предсказания и цели. «Похожесть» зависит от выбранной задачи.

Какое сходство мы хотим перенести?

В исходных данных

Слова встречаются в похожих контекстах.

«кошка спит»
«собака спит»

В представлении

Векторы получают большой скалярный отклик.

$$\mathbf{v}_w^\top \mathbf{u}_c$$

В Word2Vec $\mathbf{v}_w$ – вектор слова, $\mathbf{u}_c$ – вектор контекста.

Для классификации изображений цель другая: разные изображения кошек должны вести к одному классу.

Линейный слой сравнивает с шаблоном

$$s_k = \mathbf{w}_k^\top \mathbf{h} + b_k$$

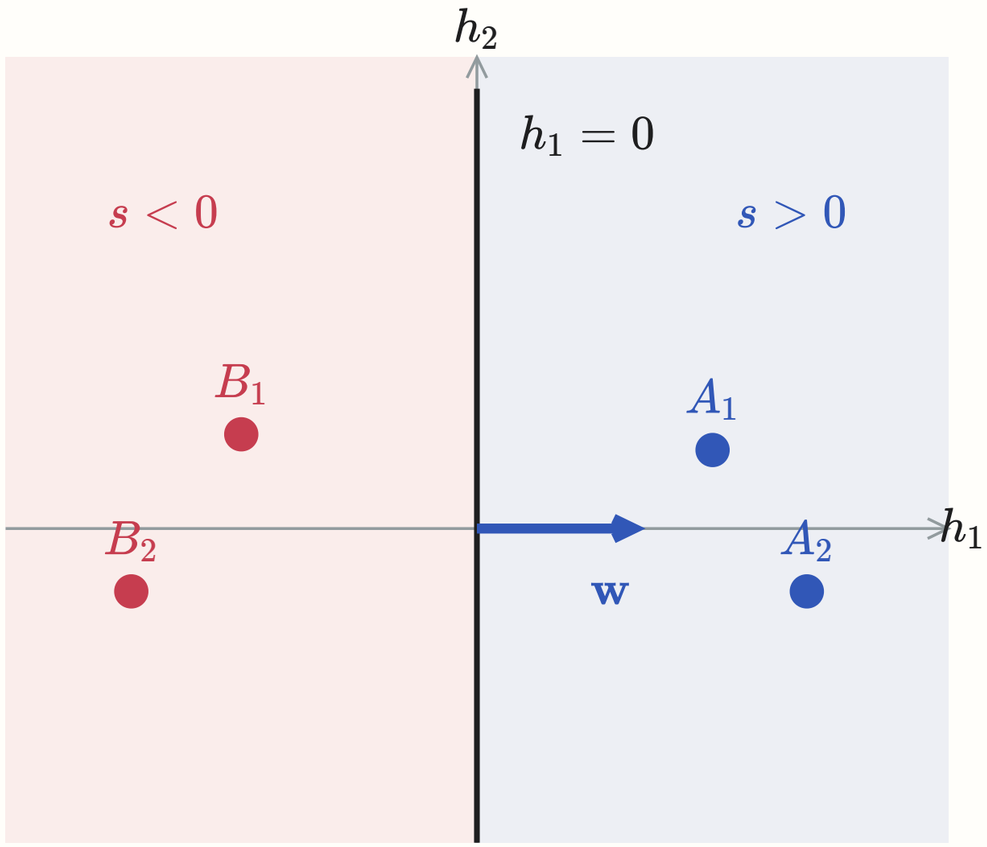
$\mathbf{h}$ – признаки объекта; $\mathbf{w}_k$ – обучаемые веса для класса k.

$$\mathbf{w}^\top \mathbf{h} = \|\mathbf{w}\|_2 \|\mathbf{h}\|_2 \cos\alpha$$

s_k – оценка класса; b_k – смещение; α – угол между векторами.

При фиксированных длинах отклик максимален у совпадающих направлений. Без нормировки важны и длины.

Нормаль задаёт направление роста скора



Два класса в пространстве признаков.

$$\mathbf{w} = (1, 0)^\mathsf{T}, \quad b = 0$$

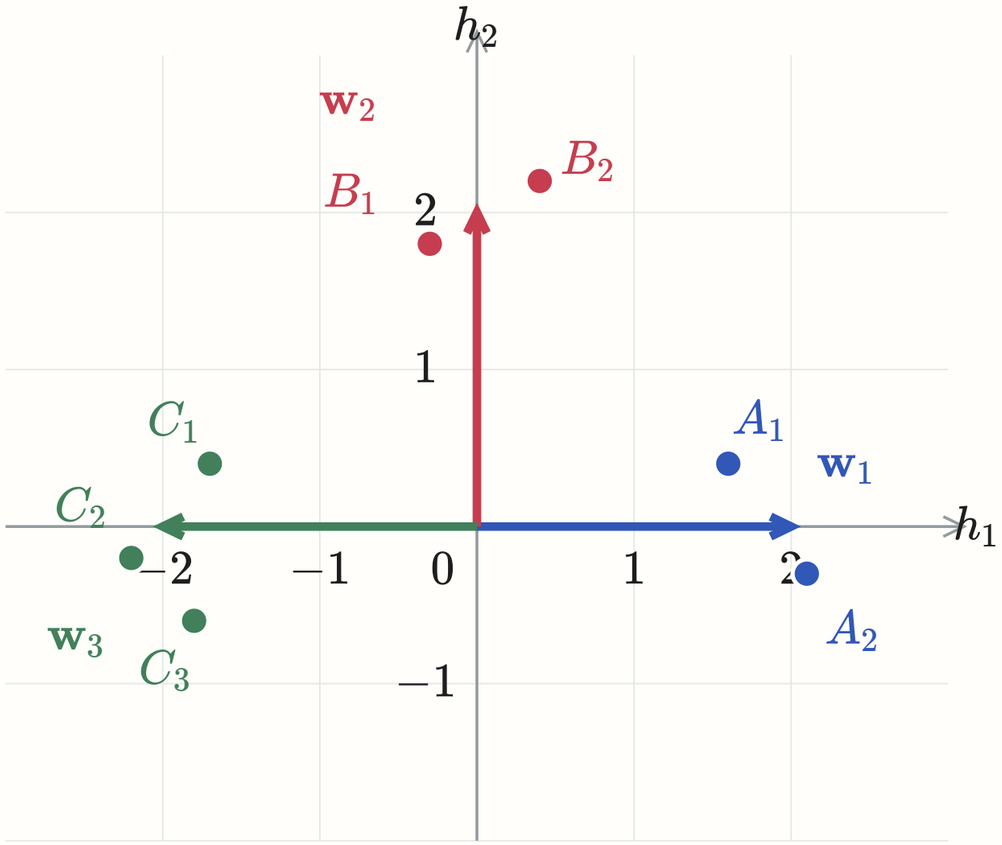
$$s = \mathbf{w}^\mathsf{T}\mathbf{h} = h_1$$

Справа – класс 1. Слева – класс 0.

Граница s = 0 перпендикулярна вектору w.

Учебные точки и заданные веса.

Три класса – три вектора весов



Семь объектов: 2 синих, 2 красных, 3 зелёных.

$$\mathbf{w}_1 = (2, 0)^\mathsf{T}$$

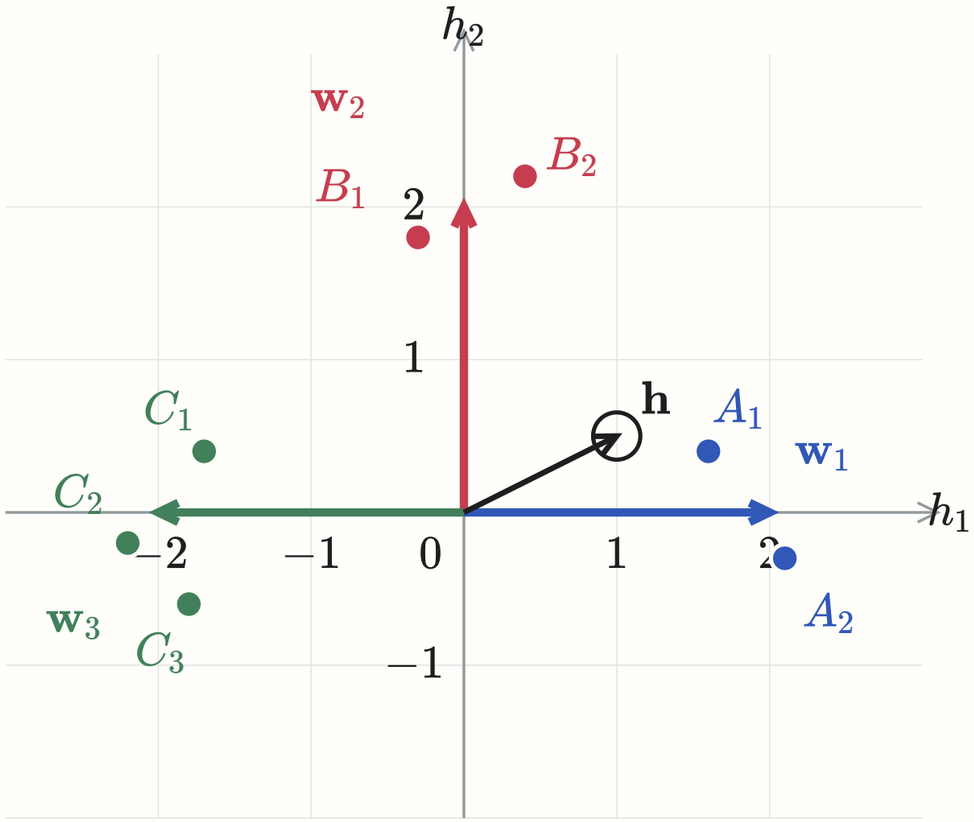
$$\mathbf{w}_2 = (0, 2)^\mathsf{T}$$

$$\mathbf{w}_3 = (-2, 0)^\mathsf{T}$$

Каждый вектор задаёт свой скор. Выбрать класс с наибольшим скором.

Здесь $b_1 = b_2 = b_3 = 0$; веса заданы для примера.

Что считает умножение на матрицу?



Строки $\mathbf{W}$ – векторы весов трёх классов.

$$\underbrace{\begin{pmatrix} 2 & 0 \\ 0 & 2 \\ -2 & 0 \end{pmatrix}}_{\mathbf{W}} \underbrace{\begin{pmatrix} 1 \\ 0.5 \end{pmatrix}}_{\mathbf{h}} = \underbrace{\begin{pmatrix} 2 \\ 1 \\ -2 \end{pmatrix}}_{\mathbf{s}}$$

Максимальный скор: класс 1.

Цвет строки $\mathbf{w}_k^\mathsf{T}$ совпадает с цветом скора s_k и класса.

Скор ещё не вероятность

Для того же объекта получены три скора:

$$\mathbf{s} = (2,\ 1,\ -2)^{\mathsf{T}}$$

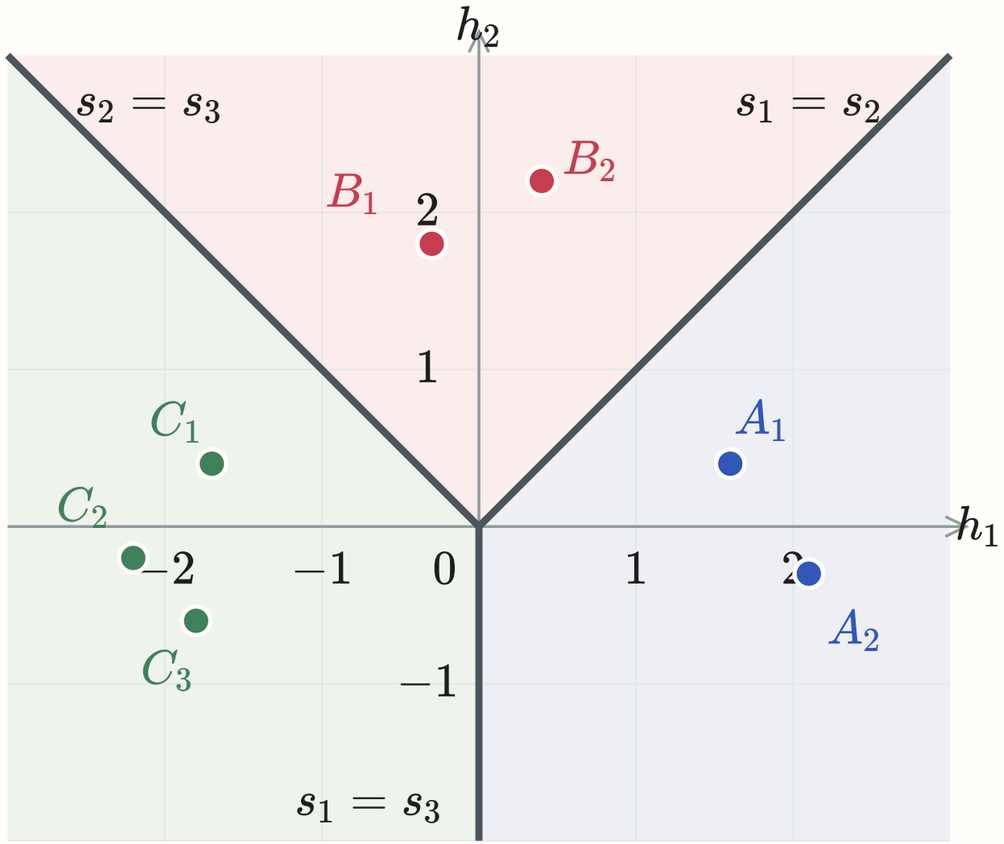
$$p_k = \frac{\exp(s_k)}{\sum_{j=1}^{3} \exp(s_j)}$$

$$\mathbf{p} \approx (0.7214,\ 0.2654,\ 0.0132)^{\mathsf{T}}$$

Побеждает класс 1. Положительных скоров может быть несколько.

Скоры перед softmax – логиты. Все классы нормируются вместе.

Какой шаблон победил в каждой области?



В каждой точке выбирается класс с наибольшим скором.

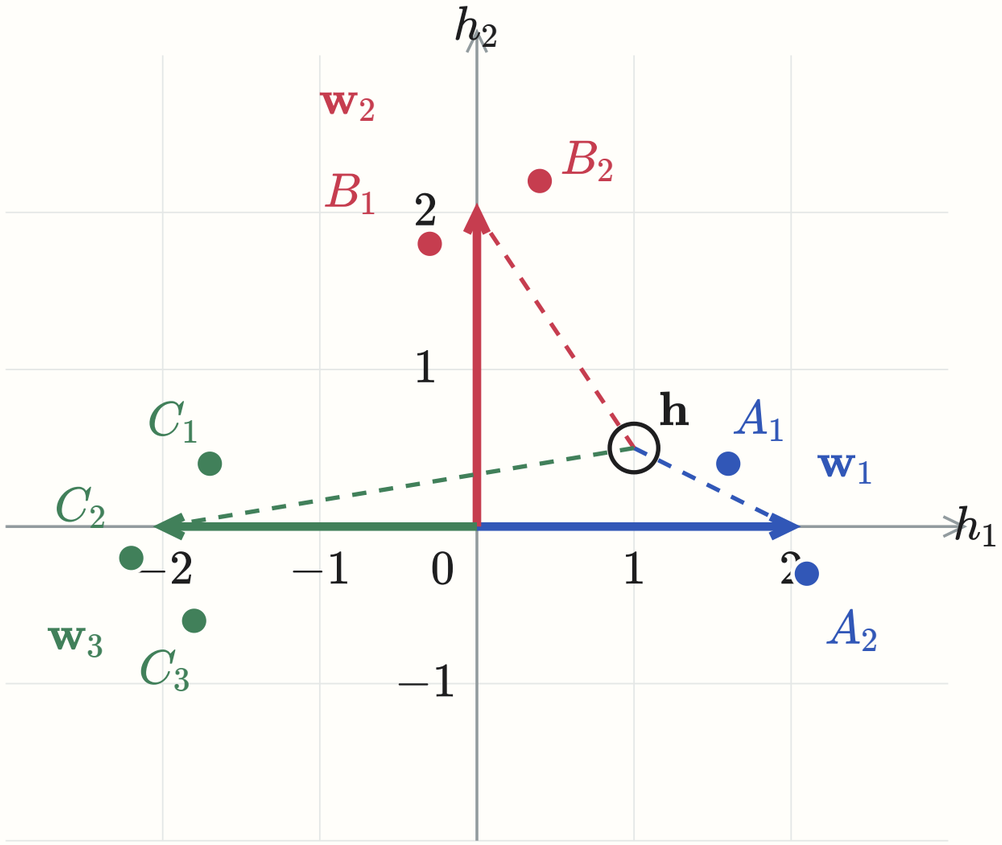
$$\hat{y} = \arg\max_{k \in \{1,2,3\}} s_k$$

Цвет показывает выбранный класс.

На границе два наибольших скора равны.

Те же три шаблона и те же семь объектов.

Когда это «ближайший шаблон»?



В этом примере длины всех трёх весов равны 2.

$$\|\mathbf{h} - \mathbf{w}_k\|^2 = \|\mathbf{h}\|^2 + 4 - 2s_k$$

d_k – расстояние от объекта до шаблона $\mathbf{w}_k$.

$$(s_1, s_2, s_3) = (2, 1, -2)$$

$$(d_1^2, d_2^2, d_3^2) = (1.25, 3.25, 9.25)$$

Наибольший скор ↔ ближайший шаблон.

Обученные веса не обязаны быть средними векторами классов.

От объекта к предсказанию

Объект уже задан своим признаковым описанием.

Объект $\mathbf{x}$ $\equiv$ **Исходное признаковое описание** $\mathbf{h} = \mathbf{x}$ $\rightarrow$ **Классификатор** $\mathbf{s} = \mathbf{W}\mathbf{h} + \mathbf{b}$ $\rightarrow$ **Предсказание** $\hat{y}$

$\mathbf{x}$ – исходный объект; $\mathbf{h}$ – то же описание на входе классификатора.

Классификатор работает с исходными признаками.

Как научиться строить другие признаки?

Можно обучать и само построение признаков

Построить новые признаки преобразованием с параметрами $\boldsymbol{\alpha}$.

Объект

$\mathbf{x}$

→

Обучаемый энкодер
(преобразование признаков)

$$\mathbf{h} = g_{\boldsymbol{\alpha}}(\mathbf{x})$$

→

Классификатор

$$\mathbf{s} = \mathbf{W}\mathbf{h} + \mathbf{b}$$

→

Предсказание

$\hat{y}$

или

$$p(y \mid \mathbf{x}; \boldsymbol{\omega})$$

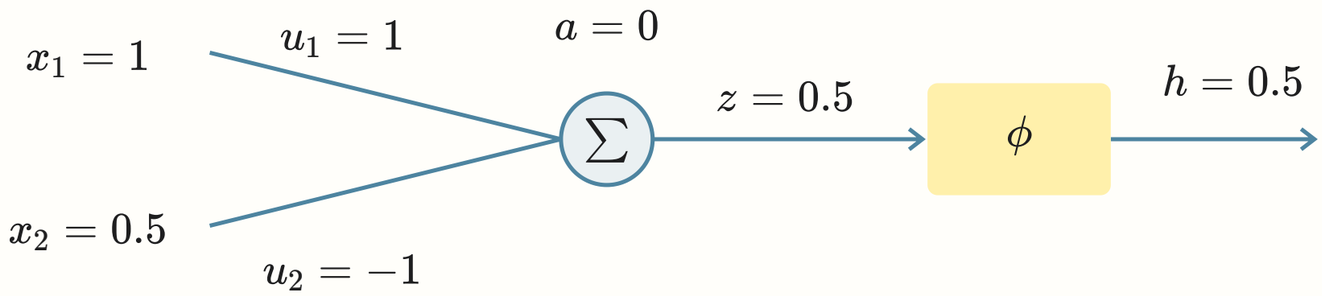
$\boldsymbol{\alpha}$ – параметры извлекателя; $\mathbf{W}, \mathbf{b}$ – параметры классификатора; $\boldsymbol{\omega} = (\boldsymbol{\alpha}, \mathbf{W}, \mathbf{b})$ – все параметры.

Обучать оба блока по одной функции потерь для конечной задачи.

Нейросеть задаёт извлекатель цепочкой простых преобразований.

Один нейрон строит один скрытый признак

Скрытый (латентный) признак вычисляется внутри модели.
Вход: $\mathbf{x} = (1,\ 0.5)^{\mathsf{T}}$. Сначала взвешенная сумма, затем функция активации.



$$z = \mathbf{u}^{\mathsf{T}}\mathbf{x} + a = 1 \cdot 1 + (-1) \cdot 0.5 + 0 = 0.5$$

$$h = \phi(z), \qquad \phi(z) = \max(0, z) \quad (\mathrm{ReLU})$$

Нейрон – аффинное преобразование и функция активации.

Несколько нейронов образуют слой

Каждая строка $\mathbf{U}$ задаёт свой вектор весов. $\mathbf{a} = \mathbf{0}$ – вектор смещений.

$$\mathbf{U} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \\ 1 & 1 \end{pmatrix} \qquad \mathbf{x} = \begin{pmatrix} 1 \\ 0.5 \end{pmatrix} \quad \rightarrow \quad \mathbf{z} = \mathbf{U}\mathbf{x} + \mathbf{a} = \begin{pmatrix} 0.5 \\ -0.5 \\ 1.5 \end{pmatrix}$$

$$z_1 = x_1 - x_2, \qquad z_2 = -x_1 + x_2, \qquad z_3 = x_1 + x_2$$

Три линейных отклика из двух координат. Ранг не превышает 2.

Строки задают обучаемые направления; они не обязаны образовывать базис.

Активация меняет каждый признак отдельно

Применить одну и ту же функцию ϕ к каждому числу в $\mathbf{z}$.

$$\mathbf{z} = \begin{pmatrix} 0.5 \\ -0.5 \\ 1.5 \end{pmatrix} \qquad \phi(z) = \max(0, z) \qquad \mathbf{h} = \phi(\mathbf{z}) = \begin{pmatrix} 0.5 \\ 0 \\ 1.5 \end{pmatrix}$$

→

Отрицательный отклик стал нулём. Положительные сохранились.

Размерность осталась той же. $\mathbf{h}$ – новые признаки для следующего слоя.

Теперь работаем с данными в новых признаках

Теперь шаблон каждого класса живёт в трёхмерном пространстве $\mathbf{h}$.

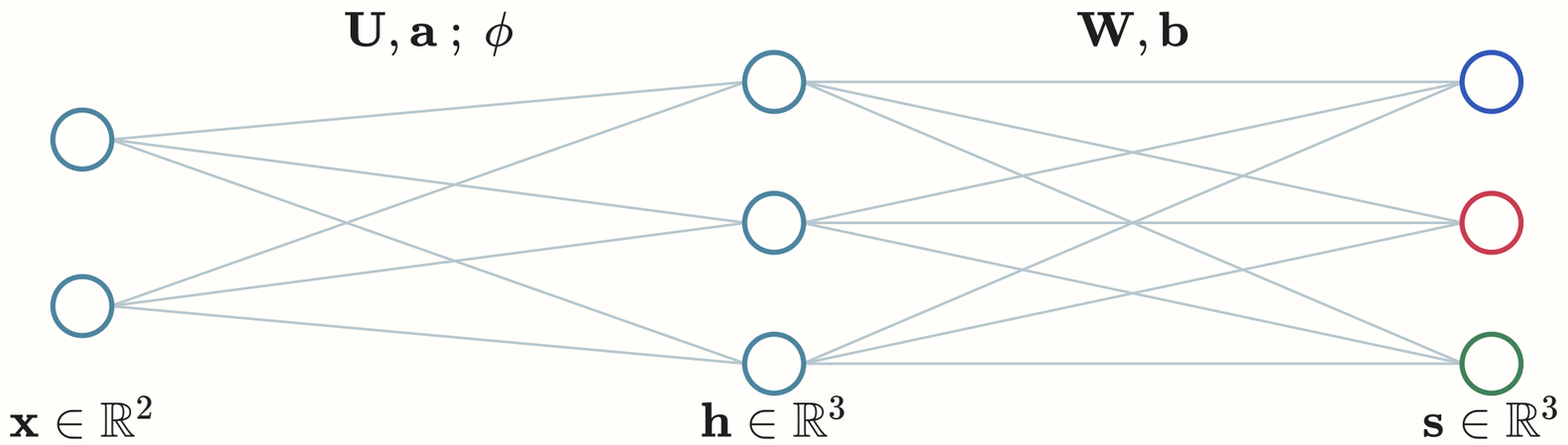
$$\mathbf{W} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad \mathbf{h} = \begin{pmatrix} 0.5 \\ 0 \\ 1.5 \end{pmatrix}, \quad \mathbf{b} = \mathbf{0}$$

$$\mathbf{s} = \mathbf{W}\mathbf{h} + \mathbf{b} = \begin{pmatrix} 1 \\ 0 \\ 3 \end{pmatrix}, \qquad \hat{y} = 3$$

Строка весов → скор класса. Побеждает наибольший скор.

Скрытый слой строит признаки; выходной слой использует их для задачи.

Как читать схему сети?



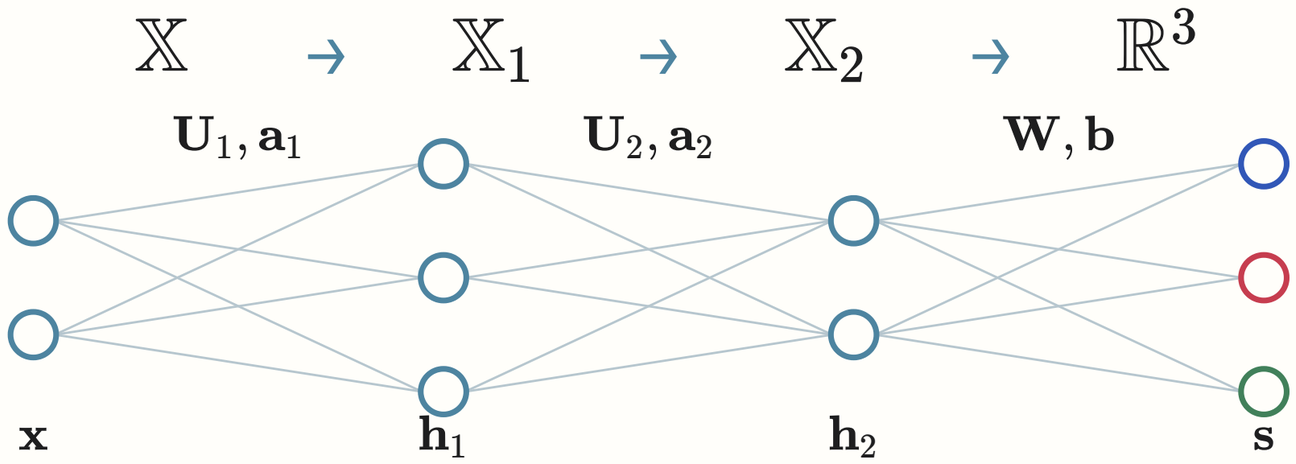
Кружок – одно число. Связи – веса, которые входят во взвешенную сумму.

$$\mathbb{X} \xrightarrow{\mathbf{U},\mathbf{a}} \mathbb{Z}_1 \xrightarrow{\phi} \mathbb{X}_1 \xrightarrow{\mathbf{W},\mathbf{b}} \mathbb{R}^3$$

Здесь два обучаемых преобразования и один скрытый слой признаков.

$\mathbf{x} \in \mathbb{X}, \mathbf{z} \in \mathbb{Z}_1, \mathbf{h} \in \mathbb{X}_1; \mathbb{Z}_1$ – пространство до активации.

Добавим ещё один скрытый слой



φ – поэлементная функция активации.

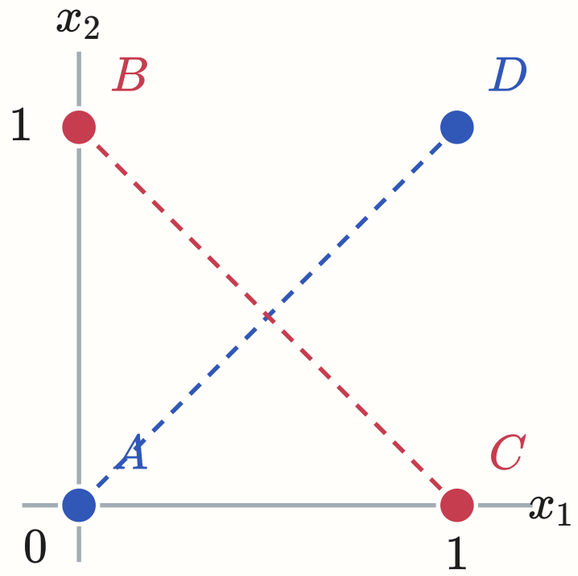
$$\mathbf{h}_1 = \phi(\mathbf{U}_1\mathbf{x} + \mathbf{a}_1), \qquad \mathbf{h}_2 = \phi(\mathbf{U}_2\mathbf{h}_1 + \mathbf{a}_2)$$

$$\mathbf{s} = \mathbf{W}\mathbf{h}_2 + \mathbf{b}, \qquad \hat{y} = \arg\max_k s_k$$

Сеть учит признаки и шаблоны последнего слоя вместе.

Кружки – отдельные числа. Схема 2 → 3 → 2 → 3.

Зачем нужны функции активации? Пример XOR



Учебная бинарная задача XOR: одинаковые цвета в противоположных углах.

Синий – класс 0; красный – класс 1.

Одна прямая эти классы не разделит.

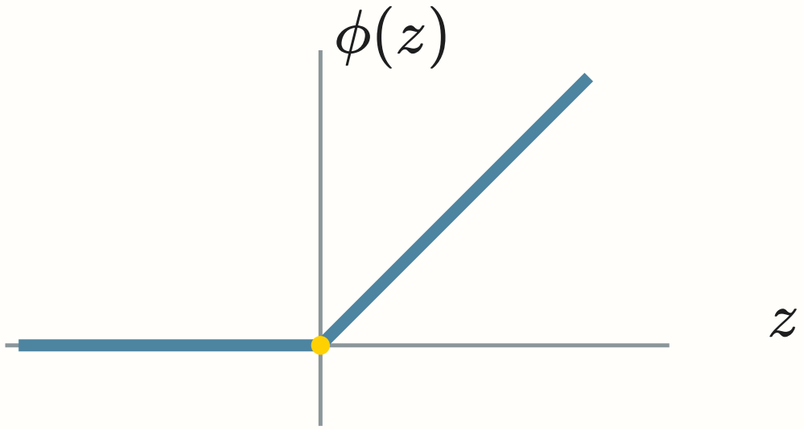
$$\mathbf{U}_2(\mathbf{U}_1\mathbf{x} + \mathbf{a}_1) + \mathbf{a}_2 = \mathbf{A}\mathbf{x} + \mathbf{c}$$

Каскад без активаций остаётся аффинным.

Меняется запись, но не класс отображений.

Пример функции активации ReLU

$$\phi(z) = \max(0, z)$$



$$\phi(1) + \phi(-1) = 1$$
$$\phi(1 + (-1)) = 0$$

Сложение не сохраняется.

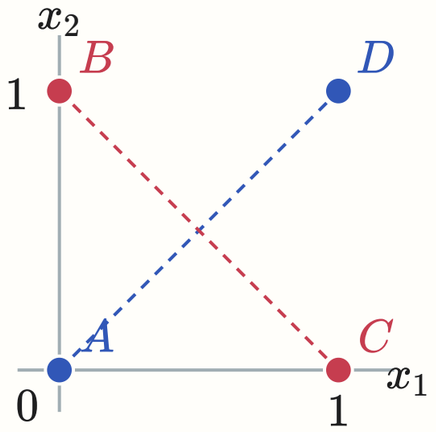
Правило меняется в нуле. Функция непрерывна, но нелинейна.

В новых признаках задача линейно разделима

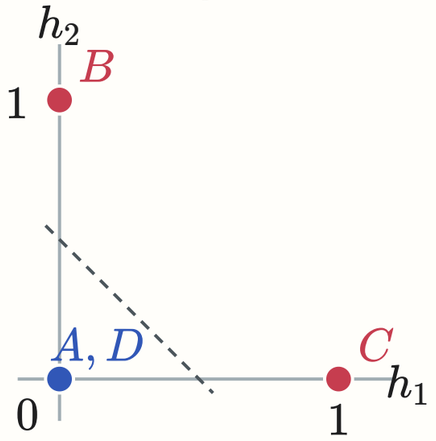
ReLU: $\phi(z) = \max(0, z)$. Синий – класс 0; красный – класс 1.

$$h_1 = \phi(x_1 - x_2), \qquad h_2 = \phi(x_2 - x_1)$$

Исходные признаки

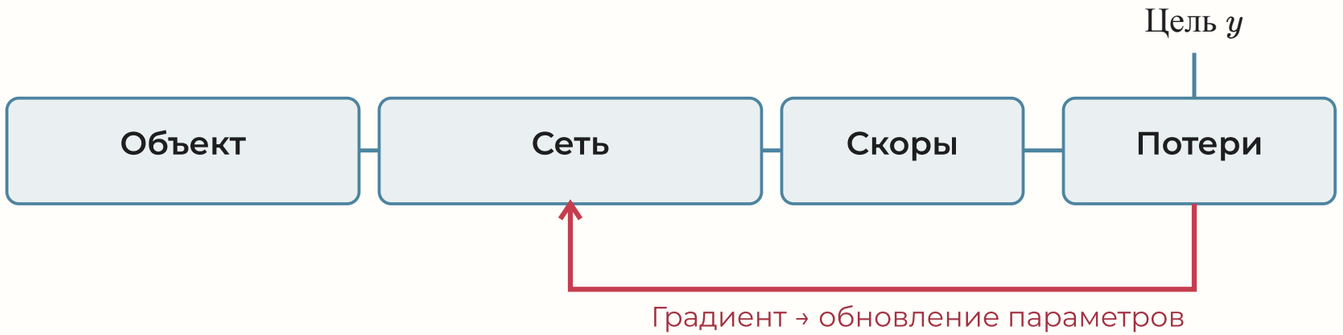


Новые признаки



Класс 1, если $h_1 + h_2 > 0.5$. Теперь хватает прямой.

Как найти параметры преобразований?



ℓ – функция потерь; θ – все веса и смещения; η > 0 – размер шага.

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \ell$$

Обратное распространение вычисляет градиент по цепному правилу.

Существование подходящих весов ещё не гарантирует, что обучение их найдёт.

Как цепное правило проходит через выход?

Бинарный выход: $q = \frac{1}{1+e^{-t}}, \quad t = \mathbf{w}^{\mathsf{T}}\mathbf{h} + b$

$y \in \{0, 1\}$ – целевая метка; q – вероятность класса 1.

$$\ell = -y \log q - (1 - y) \log(1 - q)$$

$$\frac{\partial \ell}{\partial q} = -\frac{y}{q} + \frac{1 - y}{1 - q}, \qquad \frac{\partial q}{\partial t} = q(1 - q)$$

$$\frac{\partial \ell}{\partial t} = q - y$$

Обратный проход вычисляет градиент; оптимизатор делает шаг.

Одна активация – две роли

Прямой проход

$$h = \phi(z)$$

Активация меняет представление объекта.

У ReLU справа наклон 1, слева – 0.

Обратный проход

$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial h}\,\phi'(z)$$

Производная влияет на градиент.

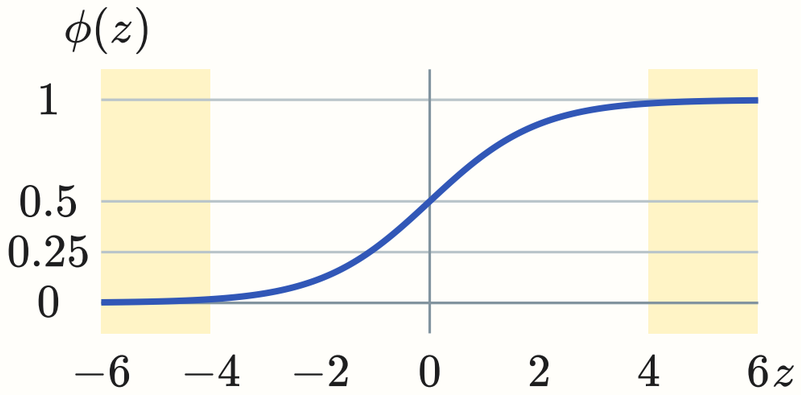
Нелинейность помогает представить ответ; её производная влияет на обучение.

Сравним формы активаций и их производных.

Sigmoid: на краях градиент ослабевает

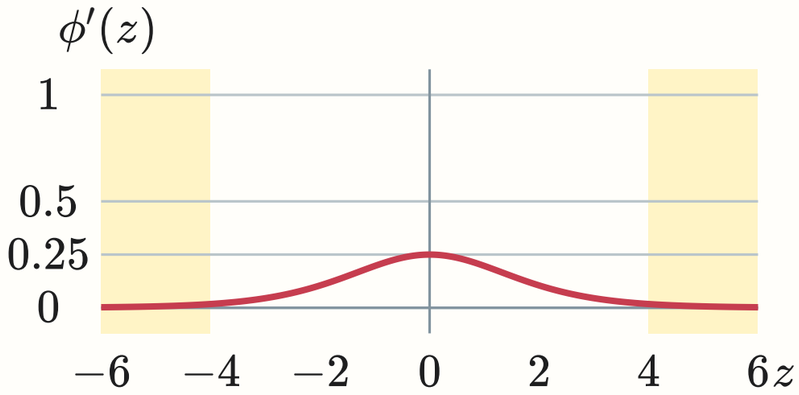
Функция

$$\phi(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Производная

$$\phi'(z) = \sigma(z)(1 - \sigma(z))$$



Выход от 0 до 1; максимальный наклон – 0.25 при z = 0.

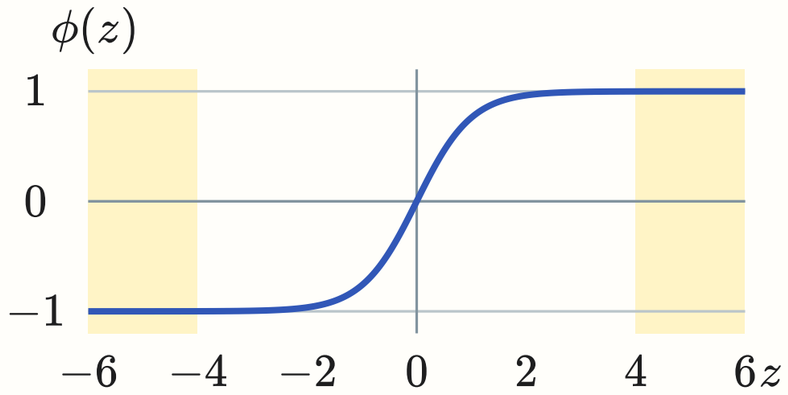
Жёлтые области: функция насыщается, производная близка к нулю.

Sigmoid полезна на бинарном выходе; в скрытых слоях важно следить за насыщением.

Tanh: знак сохраняется, насыщение тоже

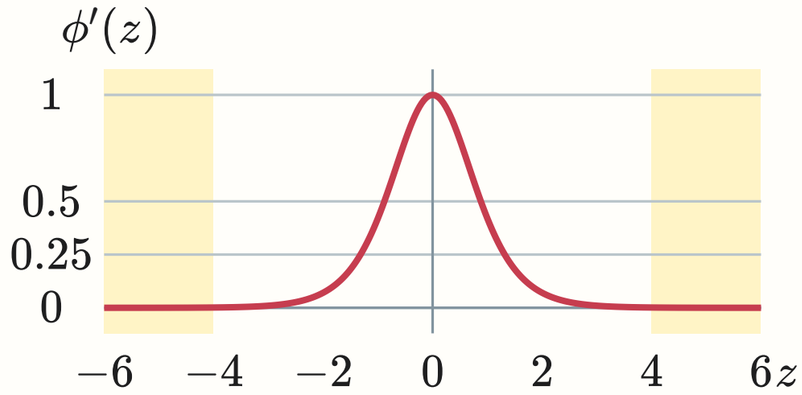
Функция

$$\phi(z) = \tanh(z)$$



Производная

$$\phi'(z) = 1 - \tanh^2(z)$$



Tanh – гиперболический тангенс; выход от −1 до 1.

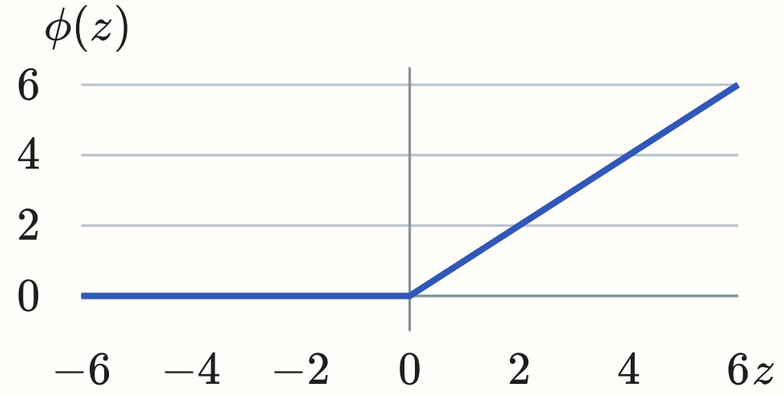
В нуле наклон – 1. Жёлтые области: насыщение и малая производная.

Нечётность функции не гарантирует нулевое среднее её выходов.

ReLU: справа градиент проходит, слева – нет

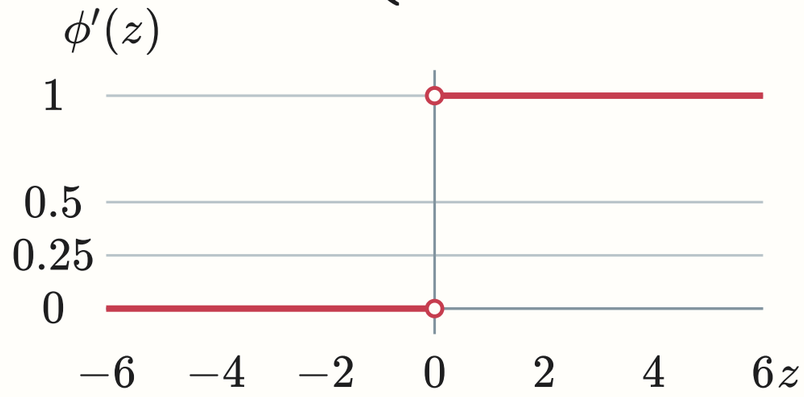
Функция

$$\phi(z) = \max(0, z)$$



Производная

$$\phi'(z) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases}$$



Справа функция не насыщается: наклон равен 1. Слева отклик и наклон равны 0.

На отрицательной стороне этот путь не передаёт градиент к входу нейрона.

В нуле обычной производной нет; для обратного прохода выбирают соглашение.

Как глубина влияет на градиент?

В обратном проходе локальные производные перемножаются.

$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial h}\,\phi'(z)$$

$$\text{Sigmoid}, z = 0: \quad 0.25^4 = \frac{1}{256} \approx 0.0039$$

$$\text{Tanh}, z = 0: \quad 1 \cdot 1 \cdot 1 \cdot 1 = 1$$

$$\text{ReLU}, z > 0: \quad 1 \cdot 1 \cdot 1 \cdot 1 = 1$$

Маленькие множители могут ослаблять сигнал; нулевой множитель обнуляет этот путь.

Только множители активаций: полный градиент содержит ещё веса и другие пути.

Активацию выбирать под роль слоя

Активация	Когда полезна форма выхода	Что проверить при обучении
Sigmoid	Значение от 0 до 1, например бинарный выход	Насыщение скрытых нейронов
Tanh	Нужен ограниченный выход обоих знаков	Насыщение при больших |z|
ReLU	Скрытый слой полносвязной или свёрточной сети	Долю постоянно неактивных нейронов

Сравнивать функцию потерь, масштабы активаций и градиентов.

Выбор активации связан с инициализацией, шагом обучения и устройством сети.

ReLU – разумная отправная точка для нашего примера, а не универсальный победитель.

Что показывает картинка скрытого нейрона?

Каждая точка – вход

$$(x_1, x_2) \longmapsto h_j(x_1, x_2)$$

Цвет в Playground – значение одного скрытого признака в этой точке.

Каждый запуск – пример

Граница зависит от признаков, архитектуры и начальных весов.

Несколько запусков не доказывают, что одна активация всегда быстрее.

Не путать карту значений признака с облаком объектов в скрытом пространстве.

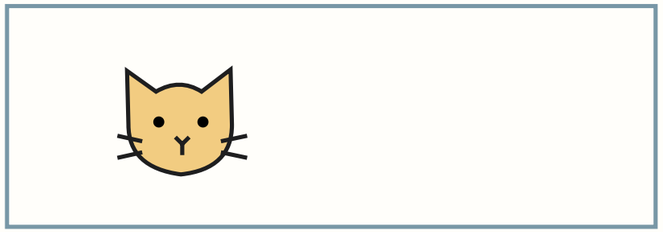
Свёрточные нейронные сети

От общих преобразований признаков – к устройству изображения.

> Локальные шаблоны и общие веса.

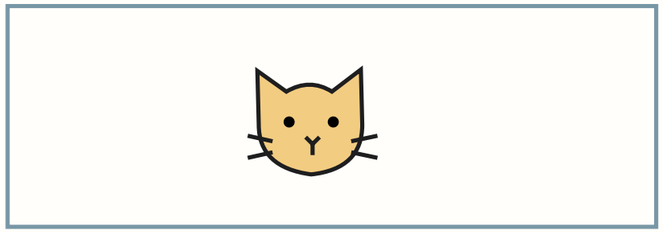
Кошка сдвинулась. Класс изменился?

До сдвига



Класс: кошка

Сдвиг вправо



Класс: кошка

Значения многих пикселей изменились. Класс остался тем же.

Нужно использовать соседство пикселей и искать один шаблон в разных местах.

Что сохраняет перестановка пикселей?

Одна фиксированная перестановка для всех объектов train и test.

$$\mathbf{x}' = \mathbf{P}\mathbf{x}, \qquad \mathbf{P}^{\mathsf{T}}\mathbf{P} = \mathbf{I}$$

$\mathbf{P}$ – перестановочная матрица; $\mathbf{W}$ – веса полносвязного слоя.

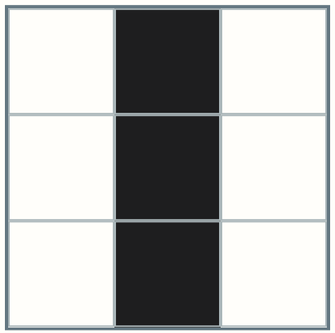
$$\mathbf{W}' = \mathbf{W}\mathbf{P}^{\mathsf{T}} \quad \Longrightarrow \quad \mathbf{W}'\mathbf{x}' = \mathbf{W}\mathbf{x}$$

Числа сохраняются. Пространственное соседство меняется.

Класс полносвязных моделей тот же; траектория случайного обучения не обязана совпасть.

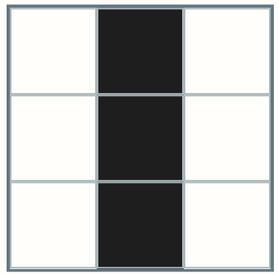
Какой шаблон узнаёт этот рисунок?

Объект



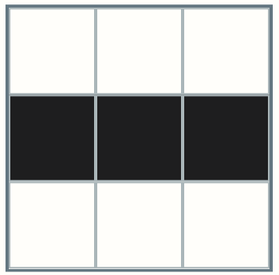
Чёрное: 1 · белое: 0

Вертикаль



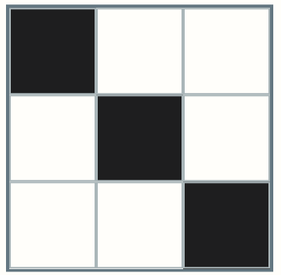
$s_1 = 3$

Горизонталь



$s_2 = 1$

Диагональ



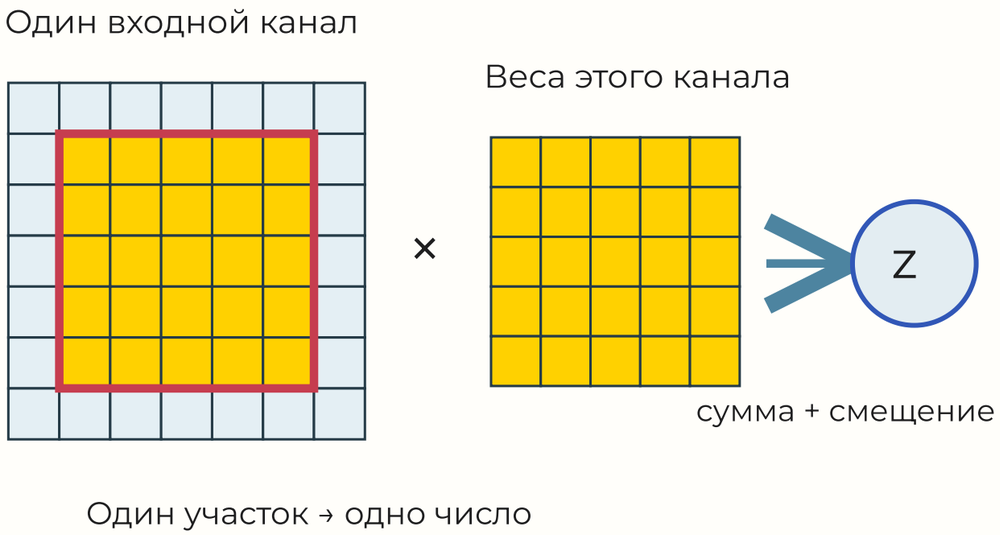
$s_3 = 1$

Отклик – сумма произведений соответствующих пикселей.

$$s_k = \mathbf{w}_k^{\mathsf{T}} \mathbf{h}, \qquad \mathbf{h}, \mathbf{w}_k \in \mathbb{R}^9$$

Три заданных шаблона 3×3; смещения равны нулю. Это учебный пример.

Как маленький участок даёт одно число?



Один канал: участок 5×5 и ядро 5×5 разворачиваем в два вектора.

$$\mathbf{x}_{u,v}, \mathbf{w} \in \mathbb{R}^{25}$$

Перемножаем соответствующие числа, складываем и добавляем смещение b.

$$z_{u,v} = \mathbf{w}^{\mathsf{T}} \mathbf{x}_{u,v} + b$$

Как одно окно заполняет карту 2×2?

Вход 4×4

1	2	1	0
0	1	2	1
2	0	1	2
1	2	0	1

Ядро

0	0	0
0	1	0
0	0	0

b = 0, шаг 1

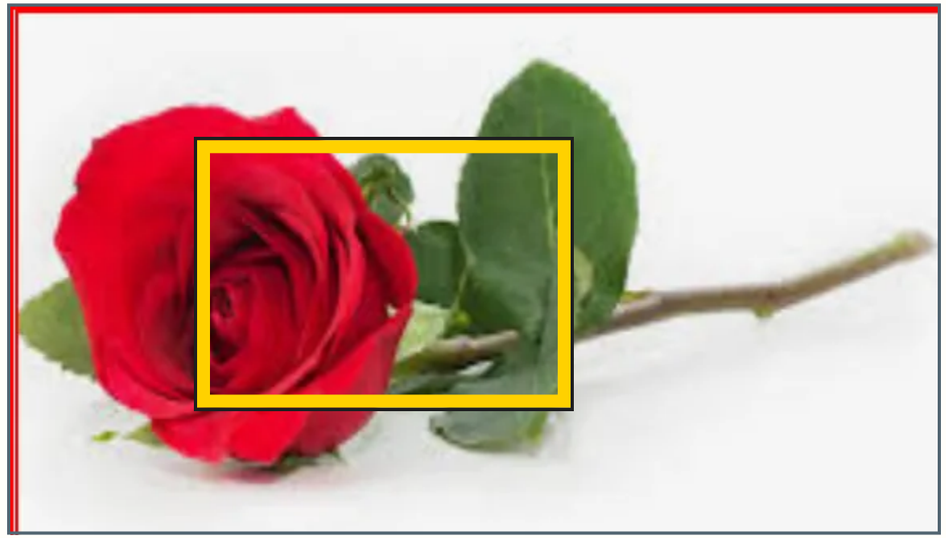
Выход 2×2

1	**2**
0	**1**

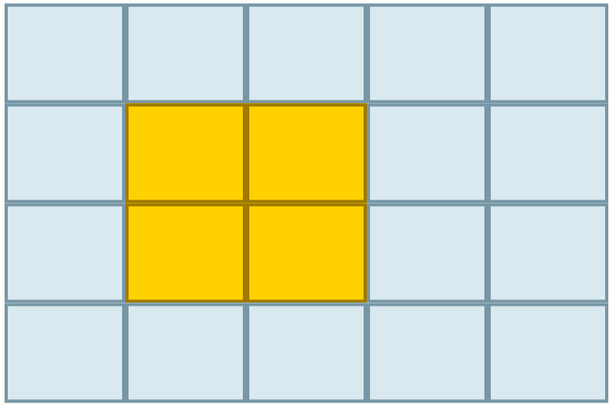
Одно и то же ядро во всех четырёх положениях. В этом примере выход равен центральному пикселю каждого окна.

Где на фотографии откликнулся фильтр?

Представим фильтр, который сильнее отвечает на границу лепестка и фона. Одни и те же веса проверяют разные участки.



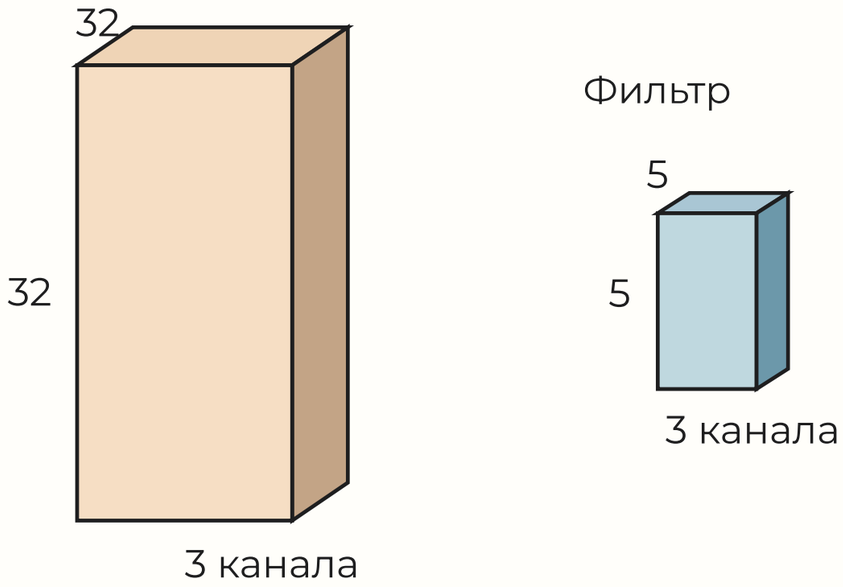
Карта откликов



Положение ярких ячеек соответствует месту найденного шаблона.

Иллюстрация условная: карта откликов нарисована для объяснения и не является результатом запуска обученной модели.

Что в изображении пространственное, а что нет?



Высота и ширина задают соседство пикселей. RGB – три значения в каждой точке.

$$X \in \mathbb{R}^{3\times 32\times 32}$$

Фильтр 5×5 охватывает все три канала.

$$W \in \mathbb{R}^{3\times 5\times 5}$$

У RGB-фильтра 3·5·5 = 75 весов. Число выходных карт задаётся числом фильтров.

Что именно суммирует свёртка?

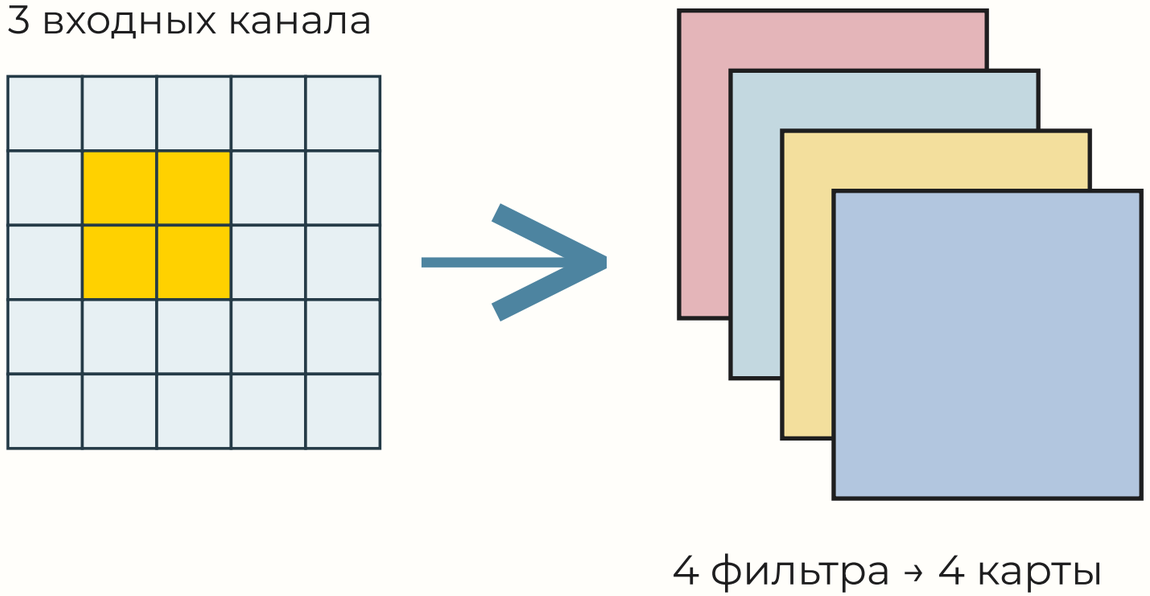
Для одного выходного канала: суммируем по окну и всем входным каналам.

$$Z_{u,v} = b + \sum_{c=1}^{C_{\text{in}}} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} W_{c,i,j} X_{c,u+i,v+j}$$

C_{in} – число входных каналов; k – сторона ядра; (u, v) – положение окна.

Здесь шаг 1 и нет дополнения границ. В ML это обычно называют свёрткой; формально записана кросс-корреляция.

Откуда возьмутся четыре карты вместо одной?



В обычной свёртке каждый фильтр охватывает **все три входных канала**.

Четыре фильтра формируют четыре выходные карты.

Выход: $Z \in \mathbb{R}^{4\times 28\times 28}$.

Параметров: $4(3 \cdot 5 \cdot 5 + 1) = 304$.

Одного шаблона кошки недостаточно

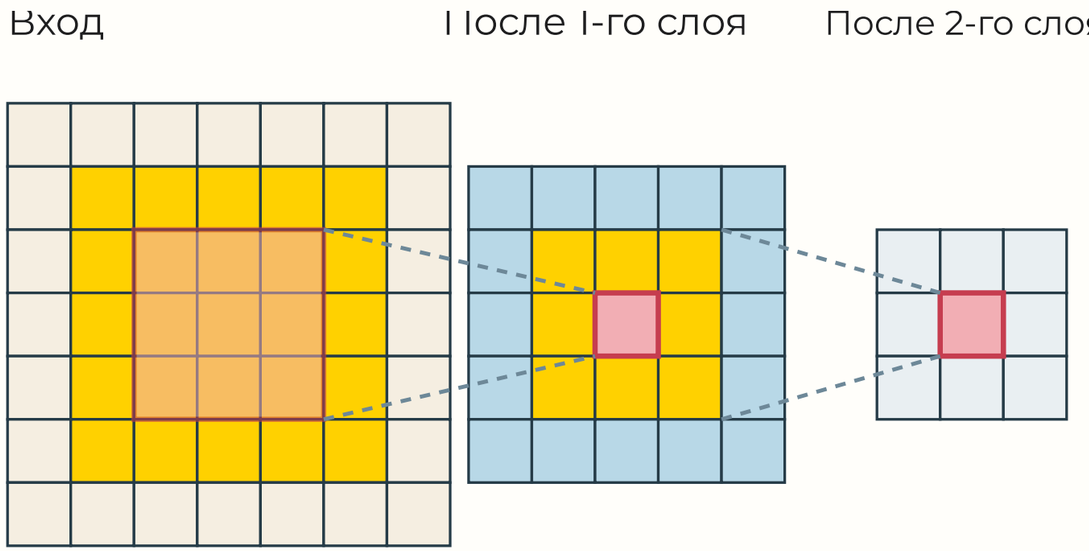
Граница → Угол / текстура → Часть объекта → Класс

Кошки бывают разного размера и в разных позах. Полезнее собрать признаки из повторяющихся элементов.

Карта откликов одного слоя становится входными каналами следующего.

Как нейрон второго слоя увидит больший участок?

Рецептивное поле – часть входа, влияющая на выбранный выходной элемент.



Два слоя 3×3 с шагом 1 → поле 5×5 на входе.

У вектора больше нет пространственных осей

$C_0 \times 7 \times 7$ → $C_1 \times 5 \times 5$ → $C_2 \times 3 \times 3$ → $C_3 \times 1 \times 1$

C_l – число каналов слоя l. Три свёртки 3×3: шаг 1, без дополнения; между ними – функции активации.

$$\mathbf{h} \in \mathbb{R}^{C_3}, \qquad \mathbf{s} = \mathbf{A}\mathbf{h} + \mathbf{b}$$

$\mathbf{A} \in \mathbb{R}^{K \times C_3}$ – веса; $\mathbf{b}, \mathbf{s} \in \mathbb{R}^K$ – смещения и оценки K классов.

Карта 1×1 – уже вектор. Но одинаковый ответ при сдвиге ещё не гарантирован.

Карта признаков – точка нового пространства

C_l, H_l, W_l – число каналов, высота и ширина слоя; vec – развёртка тензора в вектор.

$$\mathbb{X} \subseteq \mathbb{R}^{C_0 H_0 W_0}, \quad \mathbb{X}_1 \subseteq \mathbb{R}^{C_1 H_1 W_1}$$

$$\mathbf{x} = \mathrm{vec}(\mathsf{X}), \qquad \mathbf{x}_1 = \mathrm{vec}(\mathsf{H}_1)$$

Развёртка меняет запись. Пространственное соседство хранится в устройстве слоя.

Тензоры X, H_1 имеют оси: канал, высота, ширина.

Свёртку можно записать матрицей

$\mathbf{vec}$ – развёртка по строкам; $\mathbf{T_W}$ – матрица свёртки.

$$\mathbf{vec}(\mathbf{Z}) = \mathbf{T_W}\,\mathbf{vec}(\mathbf{X}) + b\mathbf{1}$$

Каждая строка выбирает одно окно. Веса ядра повторяются, остальные коэффициенты – нули.

$\mathbf{1}$ – вектор единиц: одно смещение b добавляется в каждой позиции.

Без смещения отображение линейное. Со смещением – аффинное.

Где в матрице повторяются веса фильтра?

1	4	1
1	4	3
3	3	1

Одно ядро 3×3 для всех положений окна

z_{00}	**1**	**4**	**1**	**0**	**1**	**4**	**3**	**0**	**3**	**3**	**1**	**0**	**0**	**0**	**0**	**0**
z_{01}	**0**	**1**	**4**	**1**	**0**	**1**	**4**	**3**	**0**	**3**	**3**	**1**	**0**	**0**	**0**	**0**
z_{10}	**0**	**0**	**0**	**0**	**1**	**4**	**1**	**0**	**1**	**4**	**3**	**0**	**3**	**3**	**1**	**0**
z_{11}	**0**	**0**	**0**	**0**	**0**	**1**	**4**	**1**	**0**	**1**	**4**	**3**	**0**	**3**	**3**	**1**

Выделена строка для z_{00}. Синие ячейки означают нулевой коэффициент; девять тёмных ячеек – одно и то же ядро.

- Девять весов ядра повторяются в четырёх строках матрицы, но выбирают разные пиксели входа.

Зачем между свёртками нелинейность?

$$\mathbf{T}_2(\mathbf{T}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{T}_2\mathbf{T}_1)\mathbf{x} + \mathbf{T}_2\mathbf{b}_1 + \mathbf{b}_2$$

Даже со смещениями каскад остаётся аффинным.

$$\mathbf{h}^{(l+1)} = \phi\Big(\mathbf{T}_l\mathbf{h}^{(l)} + \mathbf{b}_l\Big)$$

ReLU применяется к каждому числу: $\phi(z) = \max(0, z)$.

l – номер слоя; $\mathbf{h}^{(l)}$ – признаки; $\mathbf{T}_l, \mathbf{b}_l$ – матрица свёртки и смещения.

Что ReLU делает с картой откликов?

До ReLU: учебная карта 2×2 → **После ReLU**

$$\mathbf{Z} = \begin{pmatrix} -2 & -1 \\ 2 & 3 \end{pmatrix} \quad \rightarrow \quad \mathbf{H} = \begin{pmatrix} 0 & 0 \\ 2 & 3 \end{pmatrix}$$

$$H_{u,v} = \max(0, Z_{u,v})$$

Форма карты та же. Меняются значения, а не положения пикселей.

Из чего состоит одно преобразование?

$$\mathbb{X}_{l-1} \rightarrow \mathbb{Z}_l \rightarrow \mathbb{X}_l$$

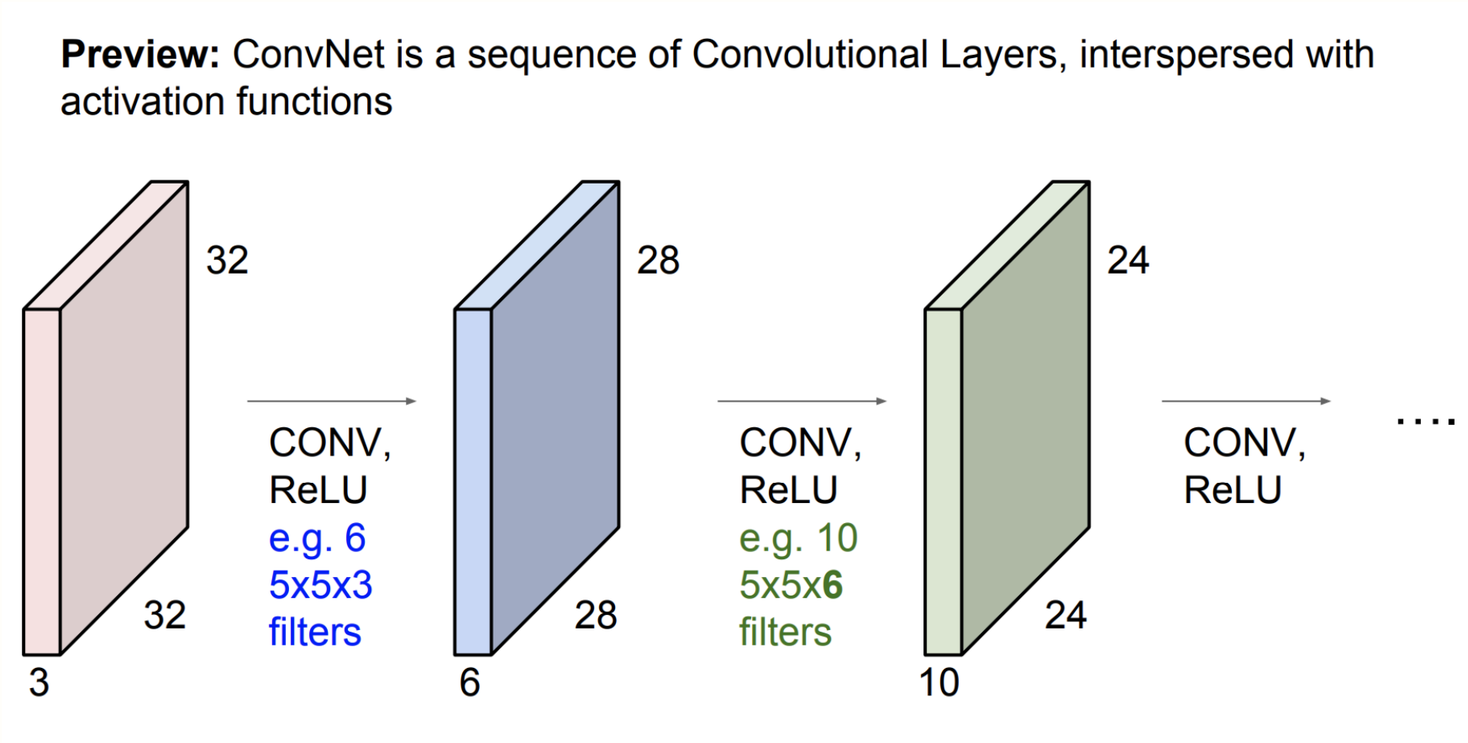
Сначала свёртка со смещением, затем поэлементная активация.

$$\mathbf{z}_l = \mathbf{T}_l \mathbf{x}_{l-1} + \mathbf{b}_l, \qquad \mathbf{x}_l = \phi(\mathbf{z}_l)$$

Общие веса задают локальный шаблон. ReLU переключает правило по значениям входа.

Если убрать активации, глубина сама по себе не добавит нелинейности.

Как карта превращается в класс?



Исходная схема: локальные свёртки и ReLU строят новые карты

Дальше получаем вектор признаков и передаём его классификатору.

Кто выбирает полезные шаблоны?

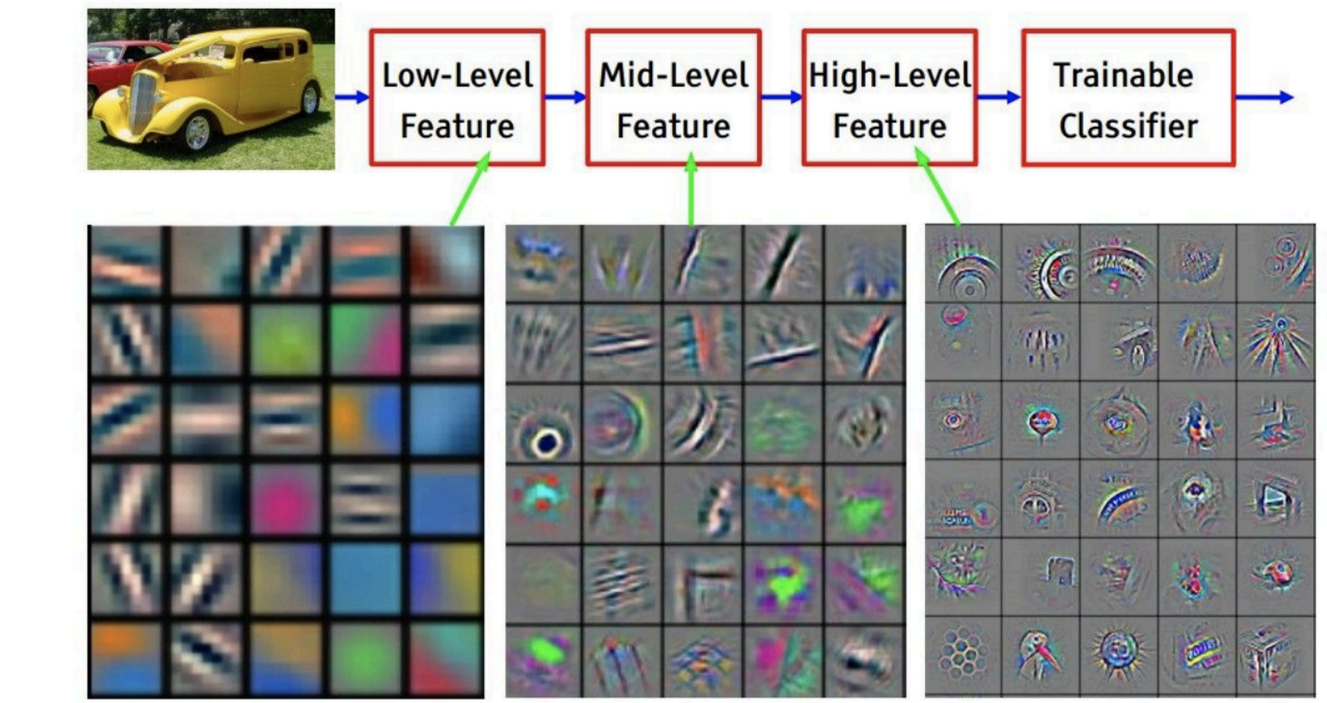
Изображение → Свёртки + активации → Классификатор → Потери

Мы задаём устройство сети и цель. Веса фильтров обучаются вместе с классификатором.

Градиенты функции потерь вычисляются по цепному правилу.

Один общий фильтр получает вклад от всех позиций, где он был применён.

От полосок – к сочетаниям признаков

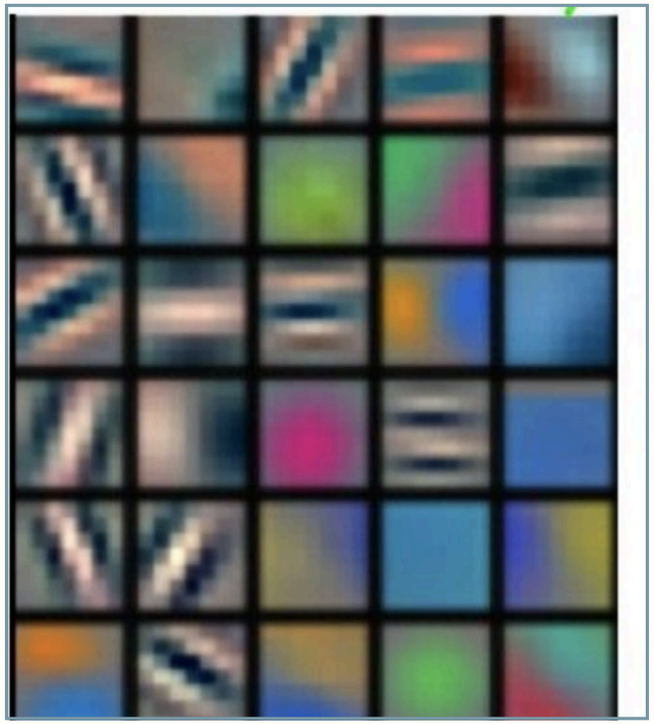


Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Иллюстрация из исходной лекции. Точный способ получения этих панелей не установлен.

Ранние признаки переиспользуются: границы полезны и для автомобиля, и для птицы.

Что здесь нарисовано?



Ранние признаки: цвет и направление



Поздние: сочетания и части объектов

Вес фильтра, карта откликов и изображение, усиливающее отклик, – три разных объекта.

Из простых элементов складывается кошка



Как в рисунке из символов: палочки и углы могут складываться в узнаваемую форму.

В сети комбинации обучаются по задаче. Эта схема – аналогия, а не буквальный словарь её фильтров.

Как показать признак из многих каналов?

Изображение → Фиксированная сеть → Отклик выбранного канала

Можно найти входы с сильным откликом или менять вход, усиливая выбранную активацию.

Получившаяся картинка объясняет отклик, но не является многоканальным тензором весов.

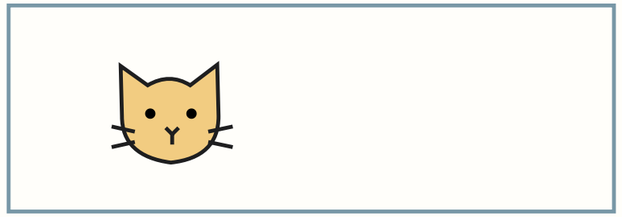
Разные методы визуализации отвечают на разные вопросы.

Инвариантность: ответ не меняется

$f : \mathbb{X} \to \mathbb{Y}$ – решение о классе; g – допустимое преобразование входа.

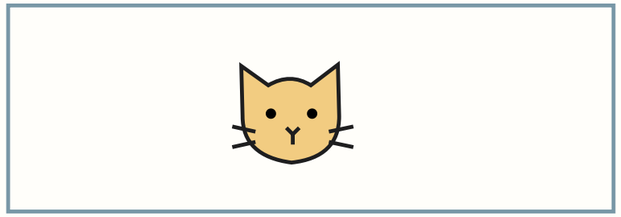
$$f(g(x)) = f(x)$$

До сдвига



Класс: кошка

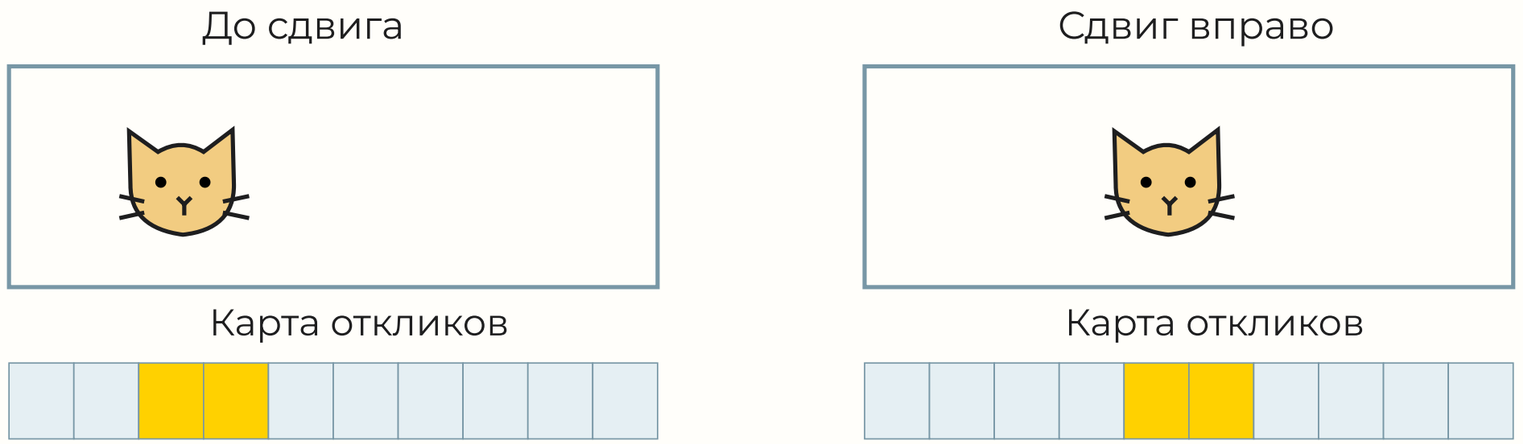
Сдвиг вправо



Класс: кошка

Желаемое свойство: ответ не меняется, пока преобразование сохраняет класс.

А что происходит с картой откликов?



Сдвигаем изображение вправо – отклик на тот же локальный признак смещается вправо.

Схема поясняет положение отклика; это не измерение обученной сети. Шаг 1, вдали от границ.

Эквивариантность: карта сдвигается вместе с входом

T_δ – сдвиг входа, T'_δ – соответствующий сдвиг выходной карты.

$$f(T_\delta x) = T'_\delta f(x)$$

Изображение	→ сдвиг →	Сдвинутое изображение
↓ свёртка		↓ свёртка
Карта откликов	→ сдвиг →	Сдвинутая карта

Для обычной свёртки с шагом 1 – на бесконечной решётке либо вдали от затронутых границ.

Почему именно свёртка?

На конечной решётке с циклическими сдвигами:

Линейность + одинаковое правило во всех позициях → циклическая свёртка.

Представим вход как взвешенную сумму единичных импульсов.

Сдвиг импульса сдвигает отклик; линейность складывает взвешенные отклики.

Для обычного конечного изображения границы требуют отдельного условия. Нелинейные функции тоже могут быть эквивариантными.

Сжали до 1×1. Ответ уже не зависит от сдвига?

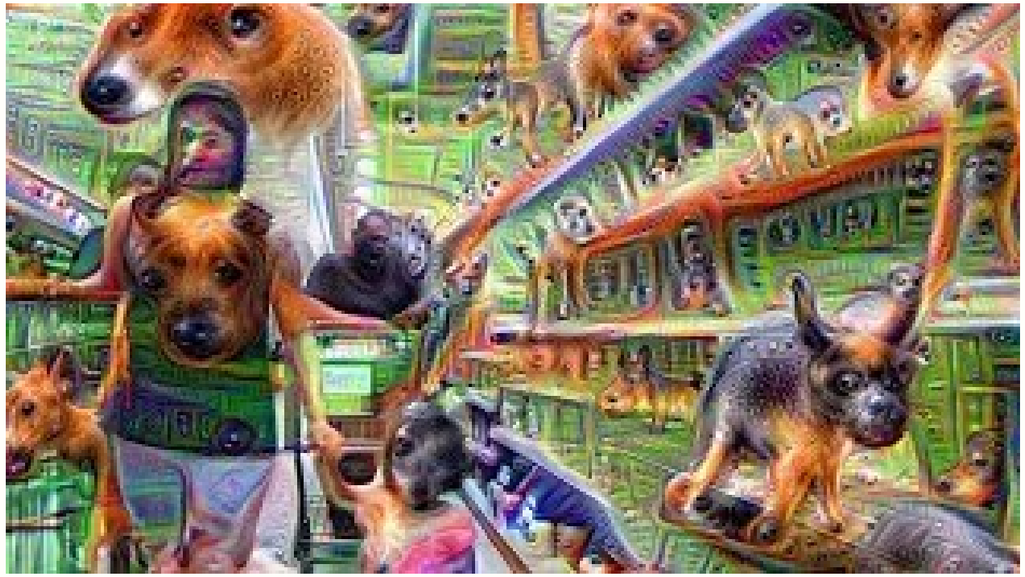
Не обязательно. При сдвиге могут измениться пиксели, попавшие в итоговое рецептивное поле.

$$f([1, 0]) = 1, \qquad f([0, 1]) = 0$$

Контрпример: фильтр [1, 0] даёт один выход. Сдвиг входа меняет это число.

Устойчивость всей сети зависит от слоёв, границ и обучения; одна форма тензора её не гарантирует.

DeepDream: усиливаем то, на что сеть откликается



Исходный DeepDream-пример

Меняем входное изображение градиентным подъёмом, увеличивая активации выбранного слоя или канала.

Повторяющиеся образы показывают, что метод **усиливает** признаки, а не доказывает, что модель «видит» объект как человек.

Теперь меняем вход, а не веса

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \eta \nabla_{\mathbf{x}} A(\mathbf{x}_t)$$

A – выбранный отклик сети; $\eta > 0$ – размер шага.

При обучении меняли параметры, уменьшая потери. Здесь фиксируем параметры и усиливаем отклик.

Обратное распространение вычисляет градиент. Оно не делает свёртку обратимой.

Зачем чайке красное пятно на клюве?



Красное пятно на клюве – вопрос из лекции

Какую задачу может решать этот заметный признак?

Кому он нужен: взрослой птице или птенцу?

Не вся птица, а важный сигнал



Тот же признак – в контексте кормления

В рассказе из лекции птенец реагирует на заметный сигнал у клюва родителя.

Для поведения может быть достаточно нескольких признаков.

Это биологическая аналогия, а не утверждение, что мозг устроен как CNN.

Сигнал может быть сильнее естественного



В рассказе из лекции: полосатый искусственный стимул вызывает особенно сильную реакцию.

Аналогия с визуализацией: «сильнее активирует» не равно «больше похоже для человека».

Схема передаёт рассказ; это не точная реконструкция исторического эксперимента.

Пять классов: что возвращает модель?

Вход: изображение 28×28 → 784 числа.

$$\mathrm{Linear}(784, 5): \quad \mathbf{x} \longmapsto \mathbf{s} \in \mathbb{R}^5$$

Для обучения

Передать logits в CrossEntropyLoss.

Цель – индекс одного из пяти классов.

Для оценки

Выбрать argmax по оси классов.

Для пакета форма выхода: [batch, 5].

Порог 0.5 применяют к вероятности в бинарной задаче, а не к первому логиту.

Опыт с перестановкой нужно закончить

На занятии возникли ошибки форм и качества. Успешного итогового сравнения нет.

1. Одна перестановка для train и test.
2. Одинаковые формы входов перед моделью.
3. Обучение и оценка используют преобразованные данные.
4. Полный запуск с начала и сохранённые результаты.

Ожидаемая эквивалентность полносвязной модели – математическое утверждение.

Причина сбоя конкретного запуска по записи окончательно не установлена.

Что мы построили?

Пиксели → Локальные шаблоны → Сочетания признаков → Вектор → ответ

Одни веса ищут шаблон в разных местах. Нелинейности позволяют собирать сложные признаки.

Свёртка переносит сдвиг в карту откликов; инвариантность ответа – отдельное требование.

Проверьте себя: почему одного линейного каскада мало и почему 1×1 не доказывает инвариантность?